CORE JAVA SERIES — DAY 8

Pillars of OOP I: Encapsulation, Inheritance, the `static` & `final` Keywords

The two pillars encapsulation and inheritance, the four access modifiers, getters and setters, packages and imports, the four types of inheritance and why multiple inheritance is banned (the Diamond Problem), the `super` keyword, and a deep dive into `static` and `final` — including why `main()` is static and what `String[] args` is for.

Encapsulation — The First Pillar

OOP mimics real-world objects: each object carries its **data** (variables) and its **behavior** (methods) as its *intrinsic* properties — an animal's legs, diet, eating and walking all belong *inside* the animal, never outside it. Encapsulation says a program must mimic this faithfully. It makes exactly **two demands**:

> **Rule 1 — Data and behavior must be encapsulated together within an object (class).** We already followed this half: every class we wrote bundles its fields and methods in one container.
>
> **Rule 2 — Never provide unrestricted access to your data.** No outsider should reach in and mutate an object's fields directly.

Why Rule 2 matters: take a `BankAccount` with a `double balance`. If `balance` is left accessible, anyone can write `ba.balance = 100000;` and assign themselves a lakh — clearly absurd. In real life a balance only changes through two *behaviors*: **deposit** and **withdraw**. Code must enforce the same thing: the user never touches `balance` directly; they only use controlled methods that validate first (e.g. "*amount ≤ balance?*" before a withdrawal).

Access Modifiers

Access modifiers are keywords that control **who has access** to a variable, method, constructor, or class. Java has **four**, and they are written immediately before the member they protect:

Modifier	Who can access	Strictness
`private`	Only within the same class. Not even subclasses or same-package classes.	Most restrictive
`default` (no keyword)	Classes in the **same package**. Applied automatically when nothing is written.	—
`protected`	Same package **+** subclasses (child classes) even in other packages.	—
`public`	Everyone, everywhere in the codebase.	Least restrictive

```
class BankAccount {
    private double balance;               // direct access blocked outside

    public void deposit(double amount)  { balance += amount; }
    public void withdraw(double amount) { if (amount <= balance) balance -= amount; }
    public double getBalance()          { return balance; }   // read-only view
}
```

> **Class-level restriction.** A top-level (root) class may only be `public` or `default` — never `private` or `protected`. `private` makes no sense ("private with respect to *what*? there is nothing above the class"), and `protected`'s child-class semantics apply to members, not whole classes. (Only a *nested* class may be `private`.) In practice `private` and `public` dominate real code; `default` and `protected` are rarer.

Note: object memory contains only the *instance* fields. Methods (static or not) are never stored inside individual objects.

Getters & Setters — Encapsulation in Practice

The standard pattern: make all fields `private`, then expose `public` methods for controlled access. `getX()` reads a field; `setX()` writes it. The win is **control** — validation lives in the setter:

```
public void setCollege(String college) {
    // validate the college actually exists (DB / API lookup) before assigning
    this.college = college;
}
public void setAge(int age) {
    // reject 200-year-old students; enforce 1..100, etc.
}
```

A field may get a getter *without* a setter (e.g. `getBalance()` with no `setBalance()`) — then it is effectively read-only. Constructors initialize starting values; getters/setters govern everything after.

Active Recall — Part 1

Q1. What two things does encapsulation demand?

A. (1) Data and behavior bundled together inside one class; (2) no unrestricted access to the data — access only through controlled public methods.

Q2. Which modifier is applied if you write no modifier at all?

A. `default` — package-level access.

Q3. Can a top-level class be declared `private` or `protected`?

A. No. Only `public` or `default`; `private`/`protected` only make sense for members (and nested classes).

Q4. What is the benefit of a getter-without-setter?

A. It exposes a read-only view of a private field — the value can be seen but never changed from outside.

Packages & import

A **package** is a folder that groups similar classes and interfaces together (java definition: *a group of similar classes and interfaces*). It lets two classes share a name — `school.Student` and `college.Student` coexist happily.

Syntax	Meaning
`import college.Student;`	Import one class from a package.
`import college.*;`	Import **all** classes of the package (the `*` wildcard).
`school.Student s2 = new school.Student();`	Fully-qualified name — skips the import entirely.
`import school.Student; import college.Student;`	**Compile error** — colliding imports; use fully-qualified names instead.

- **Built-in packages** ship with Java: `java.util` (collections — ArrayList, LinkedList…), `java.io` (input/output), `java.lang` (String etc. — **auto-imported**, never written).
- **How imports work internally:** at compile time the compiler reads each `import` line and loads the corresponding `.class` file (the compiled bytecode, e.g. `ArrayList.class`) so the JVM knows *which* class every reference means, then runs the program with zero ambiguity.
- Production packages are **hierarchical**: `com.company.project.module…` — one folder nesting inside another.

Inheritance — The Second Pillar

Real-world objects relate to each other: *a Car **is a** Vehicle, an Engineering Student **is a** Student*. This **is-a relationship** is inheritance. The parent is the **super class** (base), the child is the **sub class** (derived).

```
class Student {
    public void markAttendance() { System.out.println("Attendance marked"); }
}
class EngineeringStudent extends Student {   // is-a Student
    public void attendLab() { System.out.println("Lab attended"); }
}
```

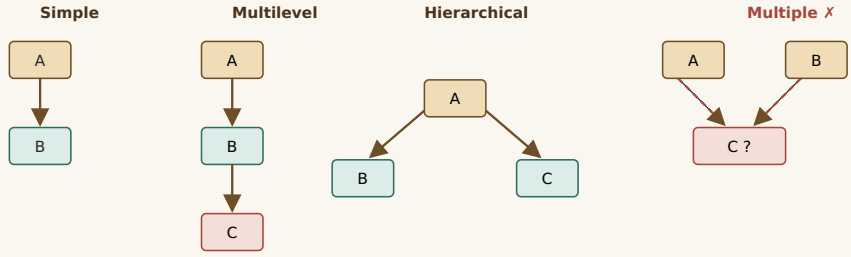
What the child gets: an `EngineeringStudent` object can call *both* `markAttendance()` (inherited) and `attendLab()` (its own). A plain `Student` object can call only `markAttendance()` — `attendLab()` on it is a compile error ("*method attendLab is undefined*").

Access through inheritance. A subclass inherits `public` and `protected` members always, and `default` members too **if both classes are in the same package**. `private` members are **never** inherited — mark a parent method `private` and even the child cannot call it.

Why inheritance matters: **Code reusability** — a common behavior like `start()` lives once in `Vehicle` and every `Car`, `Bike`, `Truck` reuses it (child-specific extras like `setGear()` stay in the child). It also **enables polymorphism** — inheritance is the prerequisite for the fourth pillar (next lectures).

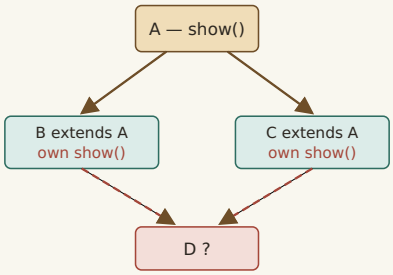
Types of Inheritance

Type	Shape	Supported in Java?
Simple	A → B	Yes
Multilevel	A → B → C (CSEStudent extends EngineeringStudent extends Student)	Yes
Hierarchical	A → B and A → C (Student → EngineeringStudent, MedicalStudent)	Yes
Multiple	A + B → C (two parents, one child)	**No — not with classes** (only via interfaces, later)



Simple, multilevel and hierarchical inheritance are supported; multiple inheritance of classes is not.

The Diamond Problem — Why Multiple Inheritance Is Banned



The Diamond Problem: which show() should D inherit — B's or C's?

Suppose it compiled: `B` and `C` both extend `A` and both **override** `show()`. Now `D` extends **both** `B` and `C` and adds nothing of its own. Calling

`d.show()` leaves the compiler unable to decide between B's version and C's version — an ambiguity that cannot be resolved at run time. Because this diamond-shaped structure produces that unresolvable ambiguity, **Java simply forbids multiple inheritance of classes**. (Interfaces provide a workaround — deferred to the interfaces lecture.)

Active Recall — Part 2

Q1. What relationship does inheritance model?

A. The is-a relationship (Car is-a Vehicle; EngineeringStudent is-a Student).

Q2. Which inherited members are never accessible in a subclass?

A. `private` members — they are not inherited at all.

Q3. Name the four inheritance types and the one Java rejects.

A. Simple, multilevel, hierarchical, multiple — multiple inheritance of classes is not supported.

Q4. State the Diamond Problem in one line.

A. If D inherits from B and C, and both override the same method from A, the compiler cannot decide which override D should use.

The super Keyword

`this` holds a reference to the **current object**; `super` holds a reference to the **parent class part** of the current object. Three uses, mirroring `this`:

Use	Form	Effect
1. Parent field	`super.x`	Access the parent's field when a same-named field exists in the child (without `super`, the child's own field wins).
2. Parent method	`super.print()`	Call the parent's version of an overridden method.
3. Parent constructor	`super(name, age, roll);`	Invoke a parent constructor from the child's constructor — must be the **first statement**.

```
class EngineeringStudent extends Student {
    EngineeringStudent(String name, int age, int roll, String college) {
        super(name, age, roll);   // parent sets name/age/roll; must be line 1
        this.college = college;
    }
    void print() {
        super.print();            // parent's print: name, age, roll
        System.out.println(college);
    }
}
```

Implicit super(). If you write no `super(...)` call, Java silently inserts a no-argument `super()` as the first line of every constructor — which is why the parent's no-arg constructor must exist (or be written) whenever a subclass is instantiated.

Active Recall — Part 3

Q1. What does super store, analogous to this?

A. A reference to the parent-class object/part — where `this` references the current object.

Q2. Where must a super(...) constructor call appear?

A. As the very first statement of the constructor.

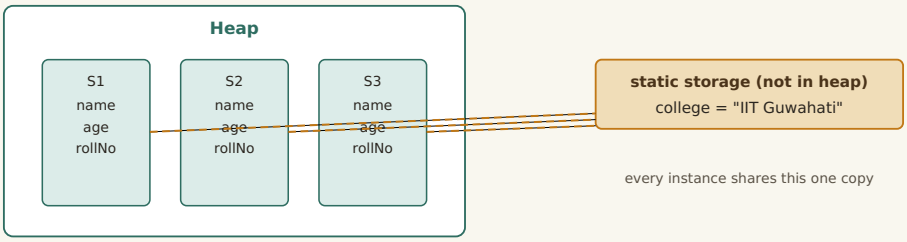
Q3. If both parent and child declare `int x;`, how do you reach the parent's x inside the child?

A. `super.x` — a bare `x` resolves to the child's field.

The static Keyword

Motivation: a hundred `Student` objects of one college would each store an identical `college` string — the same value duplicated a hundred times in the heap. But `college` is identical for every instance, so it is not an *object* property at all — it is a **class property**. `static` declares exactly that.

```
static String college;   // one shared copy for the whole class
```



Instance variables live inside each object; a static (class) variable lives in one shared place outside the heap.

- **Instance variables** (e.g. `name`, `age`, `rollNumber`) — created per object, stored in the heap, hold each object's unique state.
- **Class variables** (declared `static`) — a single copy shared by all instances; **not stored in the heap** (memory-management lecture). Change it via any reference and **every** object sees the change.

Access syntax: `Student.college = "IIT Guwahati";` — *ClassName.staticMember*. Writing `s1.college` also compiles (with a compiler warning) but the class-name form is preferred. The same applies to **static methods**: `Student.markAttendance()` works without creating a single object.

Key consequence: a static member can be used with **zero objects in existence** — which explains all the rules below.

Rules for static Methods

1. A static method can only call other static methods. A static method may be invoked with no object alive; if it then called a non-static method, the JVM would not know *on which object* to run it.

2. A static method can only access static variables. Instance fields don't exist until an object is created (`new`); a static method may run before any `new`, so `name` / `age` / `rollNumber` are simply not there to touch.

3. A static method has no access to this. `this` = reference to the *current object* — but a static method is class-level and may have no current object at all.

Static Blocks

```
static {
    college = "IIT Guwahati";   // initialize static fields here
}
```

A **static block** initializes static variables — the static counterpart of a constructor (which initializes instance variables). It runs **exactly once, when the class is loaded — before any object is created**. Instance variables can't be assigned here (rule 2); constructor-style `this.college = …` on a static field draws a warning, since `this` implies an object.

What Cannot Be static

Question	Allowed?	Why
static variable / method?	Yes	The whole point of the keyword.
static method parameter? `void print(static String name)`	**No**	Parameters are local variables living on the stack — they have no per-object existence to escape from, so static is meaningless.
static top-level class? `static class Student`	**No**	A root-level class belongs to a package, not to any object; only *nested* classes may be static.

Active Recall — Part 4

Q1. Why does changing a static field through one object affect all objects?

A. There is only one copy of a class variable, shared by every instance.

Q2. Why can't a static method use this?

A. `this` means "the current object," but a static method may run with no object in existence.

Q3. When does a static block run?

A. Once, at class-loading time — before any object is created.

Q4. Why are static method parameters forbidden?

A. Parameters are stack-local variables with no object association; making them static is meaningless.

The final Keyword

`final` before a variable turns it into a **constant** — a value that can be assigned **once** and never changed. Some values are fixed by nature: π is always 3.14, so no one should ever re-assign it.

```
class Random {
    final double PI = 3.14;     // r1.PI = 3.16; → "final field cannot be assigned"
}
```

- **Naming convention:** constants are `UPPER_SNAKE_CASE` — `PI`, `PI_VALUE`, multi-word joined by underscores.
- **Initialization:** a blank final *field* must be assigned exactly once — at declaration, in a static block (if also static), or in the constructor (allowed because the constructor runs at `new`, i.e. at the field's birth). A blank final *local* may be assigned once on a later line — Java tolerates the single deferred assignment ("you have to put *some* value in eventually"), but a second assignment fails: "*final variable may not have been assigned / already assigned*."
- **final parameters:** allowed — `void print(final String name)` locks the parameter against reassignment inside the method.
- **final methods & final classes** exist too, with different meanings (overriding and inheritance restrictions) — covered with polymorphism and inheritance in upcoming lectures.

static final — True Constants

```
static final double PI;   // or = 3.14 inline / in a static block
static { PI = 3.14; }
```

`static final` combines both ideas: one shared copy (`static`) that no one can ever change (`final`). This is the standard recipe for constants used across a whole project — `Math.PI` in the JDK is exactly this pattern.

Why main() Is static

The JVM must start executing a program *somewhere*, but it cannot create an object by itself — object creation is the programmer's job. If `main` were non-static, the JVM would have to do `Demo d1 = new Demo(); d1.main();`, which it can't. So `main` is declared `static`, letting the JVM call `Demo4.main()` directly on the class — no object needed. Corollary: **every helper method `main` calls must also be `static`** (rule 1 above) — which is why every method we've written inside the demo class carries the keyword.

```
public static void main(String[] args) { ... }
//   |      |      |         └ what args is for → below
//   |      |      └ return type: nothing
//   |      └ JVM calls Demo4.main() with no object
//   └ visible to the JVM from anywhere
```

String[] args — Command-Line Arguments

In the terminal era, programs were launched as `java Demo5` and input was supplied *at launch time*:

```
$ javac Demo5.java
$ java Demo5 Aditya Tandon Rohit Negi
Number of arguments are 4
Argument 0 is Aditya
Argument 1 is Tandon
Argument 2 is Rohit
Argument 3 is Negi
```

Every whitespace-separated token after the class name lands in the `args` array. This was originally used for **configuration** — e.g. `java Demo5 input.txt output.txt` to tell the program which files to read and write. Running with no tokens leaves `args.length == 0`. Modern relevance: framework and Spring Boot launches still pass configuration through command-line arguments, so the mechanism survives even though interactive input has moved to other channels.

Active Recall — Part 5

Q1. What does final do to a variable?

A. Makes it a constant — assignable exactly once, never re-assignable afterwards.

Q2. Where may a blank final field be initialized?

A. At declaration, in the constructor, or (if static) in a static block — exactly once overall.

Q3. Why is main() static?

A. So the JVM can invoke it via the class name without having to create an object first.

Q4. What happens to tokens typed after the class name in java Demo5 A B C?

A. They are collected into the `String[] args` array — A at index 0, B at 1, C at 2 — available inside main().